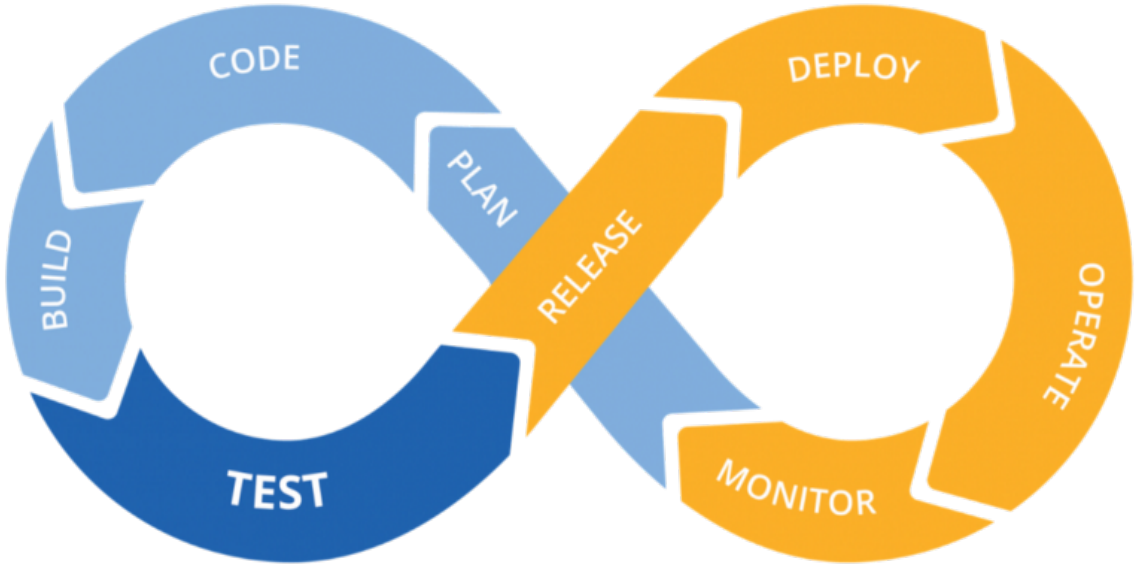


DevOps

Origine

- **Juin 2009**: John Allspaw et Paul Hammond posent les bases de DevOps lors d'une conférence: 10+ Deploys Per Day: Dev and Ops Cooperation at Flickr

- **Octobre 2009**: Le mot DevOps a été inventé par Patrick Devois durant l'organisation des premiers devopsdays en Belgique

Pourquoi le DevOps

Grosse pression du marché

On veut rapidement de nouvelles fonctions

On veut corriger plus vite les bugs voire les prévenir

On veut un SLA « 24/24 7/7 »

On veut une meilleure UX

On veut pas payer cher

On veut pouvoir grandir vite

Définition

- Combinant développement (**Dev**) et opérations (**Ops**), **DevOps** est l'union des personnes, des processus et des technologies destinés à fournir continuellement de la valeur aux clients.

- En adoptant une culture DevOps ainsi que des pratiques et outils DevOps, les équipes peuvent mieux répondre aux besoins des clients, accroître la confiance suscitée par les applications qu'elles développent, et atteindre plus rapidement les objectifs de leur entreprise.

Avantages

- Raccourcir le délai de commercialisation
- S’adapter au marché et à la concurrence
- Maintien de la stabilité et de la fiabilité du système
- Améliorer le temps moyen de récupération

Cycle de vie

- DevOps influence le cycle de vie des applications tout au long de leurs phases de planification, de développement, de livraison et d'exploitation.

- Ces phases reposent les unes sur les autres et ne sont pas spécifiques à un rôle. Au sein d'une culture DevOps, chaque rôle est, dans une certaine mesure, impliqué dans les différentes phases.

Pratiques DevOps

Intégration et livraison continue (CI / CD)

Gestion de version

Agile software development

Infrastructure en tant que code

Configuration management

Supervision continue

Intégration, livraison et déploiement continue

- L’approche **CI/CD** (continuous integration – continuous deployment/delivery) est une méthode permettant aux entreprises de renforcer l’automatisation et la surveillance dans leur processus de création de logiciels et applications.

- S’appuie sur l’utilisation d’outils performants aidant les équipes de développement et les équipes opérationnelles à mieux collaborer et suivre l’avancée de leurs travaux.

- Grâce à l’intégration, le déploiement et la distribution continus, on peut créer des applications de grande qualité et les déployer rapidement.

CI/CD

Intégration continue

Le **CI** de CI/CD désigne toujours **l'intégration continue**, à savoir un processus d'automatisation pour les **développeurs.**

Les développeurs pratiquant l'intégration continue mergent leurs changements dans la branche principale le plus souvent possible. Les changements apportés par le développeur sont validés par la création d'un build et l'exécution de tests automatisés sur celui-ci.

Vous évitez ainsi les défis liés à l'intégration susceptibles de se produire lorsque vous attendez le jour de la livraison pour merger vos changements dans la branche de version.

L'intégration continue met l'accent sur l'automatisation des tests pour s'assurer que l'app fonctionne chaque fois que de nouveaux commits sont intégrés à la branche principale.

Livraison continue

Le **CD** de CI/CD peut aussi faire référence à la **livraison continue**

La livraison continue est une extension de l'intégration continue, car elle déploie automatiquement tous les changements de code dans un environnement de test et/ou de production après l'étape de build.

Cela signifie qu'en plus des tests automatisés, vous disposez d'un processus de livraison automatisé. De plus, vous pouvez déployer votre app à tout moment en cliquant sur un bouton.

Déploiement continue

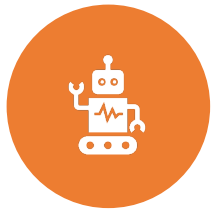
Le **CD** de CI/CD fait généralement référence au **déploiement continue**

Le déploiement continu va plus loin que la livraison continue. Grâce à cette pratique, chaque changement qui franchit toutes les étapes de votre pipeline de production est livré aux clients. Il n'y a pas d'intervention humaine, et seul l'échec d'un test empêchera le déploiement en production d'un nouveau changement

Le déploiement continu est un excellent moyen d'accélérer la boucle de feedback auprès de vos clients et de réduire la pression sur l'équipe, puisqu'il n'existe plus désormais de jour de livraison.

Les développeurs peuvent se concentrer sur le développement de logiciels, et voir leur travail lancé en production quelques minutes à peine après donné la touche finale.

Pourquoi opter pour le CI / CD



AUTOMATISATION DES TÂCHES

MEILLEURE ORGANISATION ET COLLABORATION

SURVEILLANCE EN CONTINUE

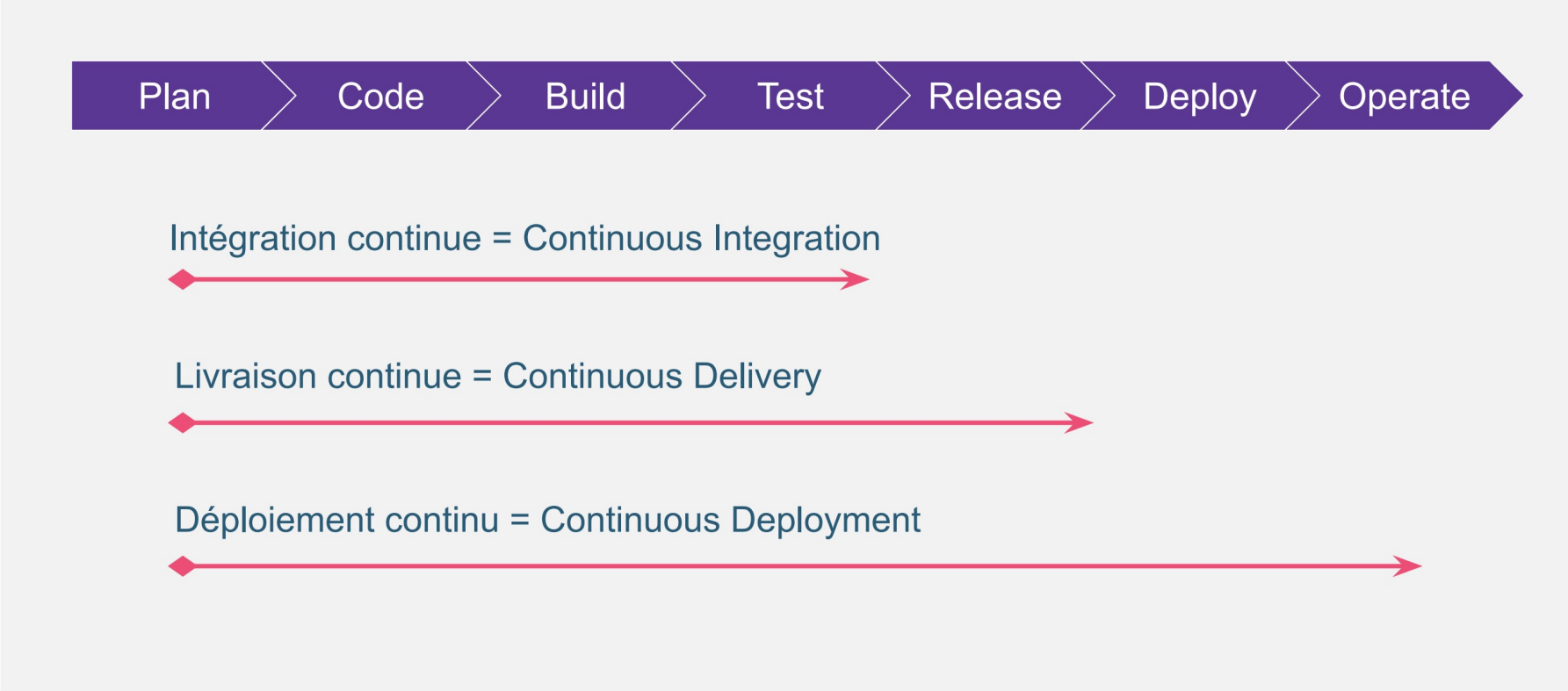
APPLICATIONS DE MEILLEURE QUALITÉ

DÉPLOIEMENT RAPIDE

SATISFACTION CLIENT

Construction et déploiement continue

L’automatisation du processus logiciel doit emmener le produit jusqu’à son environnement de production



Open Source

L’approche CI/CD est **accessible pour toutes les entreprises souhaitant travailler en DevOps.**

En effet, les principaux outils facilitant sa mise en place, comme **Gitlab, Docker et Kubernetes**, sont **tous open source**. Ils sont ainsi gratuits, libres d’accès et peuvent être librement adaptés.

Cela est particulièrement intéressant car cela **ouvre la porte à de grandes communautés de développeurs** qui vont pouvoir s’entraider et améliorer toujours plus ces outils.

CI / CD et conteneurisation

- Nous venons de parler d’outils comme Docker et Kubernetes qui reposent sur **le concept de la conteneurisation** que ce soit pour les créer ou bien les orchestrer.

- Dans l’approche CI/CD, ces outils vont permettre **un développement bien plus efficient** comparé à l’utilisation de machines virtuelles (VM).

- En effet, le principe de conteneurisation implique **une moindre consommation de ressources** car celles-ci vont être centralisées et partagées à toutes les composantes des applications.

- Cette interdépendance permet ainsi aux entreprises de **tirer profit d’architectures « cloud-based »** qui, en plus d’être dans l’ère du temps, optimisent l’organisation et la gestion des ressources.

Outils CI/CD

Jenkins

Travis CI

circleci

CI/CD

Cloud Providers

Google Cloud

netlify

Pipeline

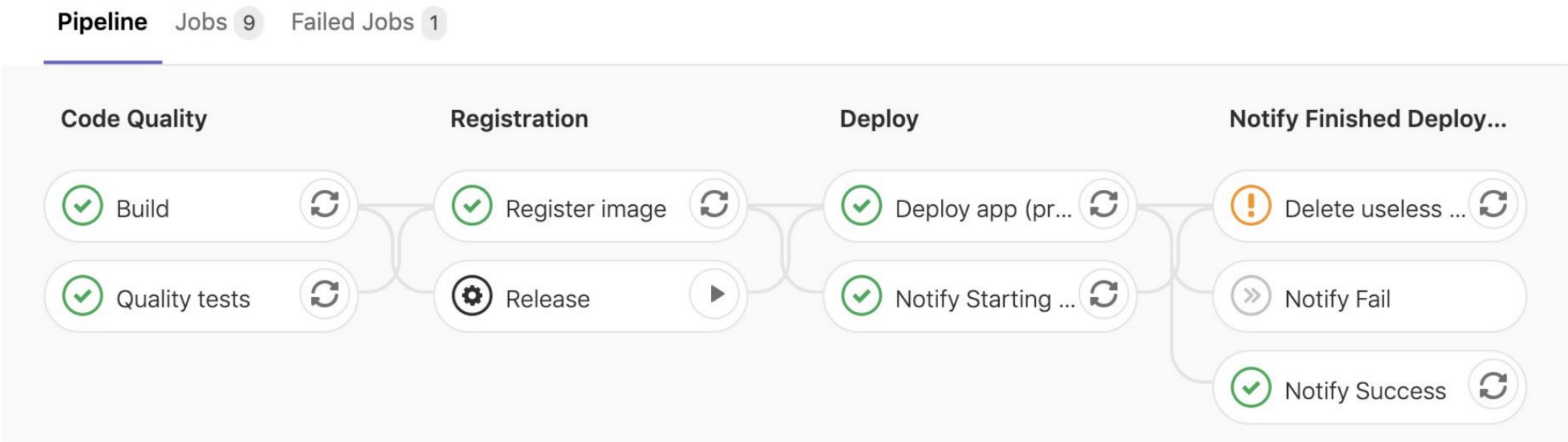
La création d’une feature va passer par différentes étapes qui sont exécutées à la suite et de manière répétitive :

- Conception
- Développement
- **Build**
- **Exécution des tests**
- **Déploiement**

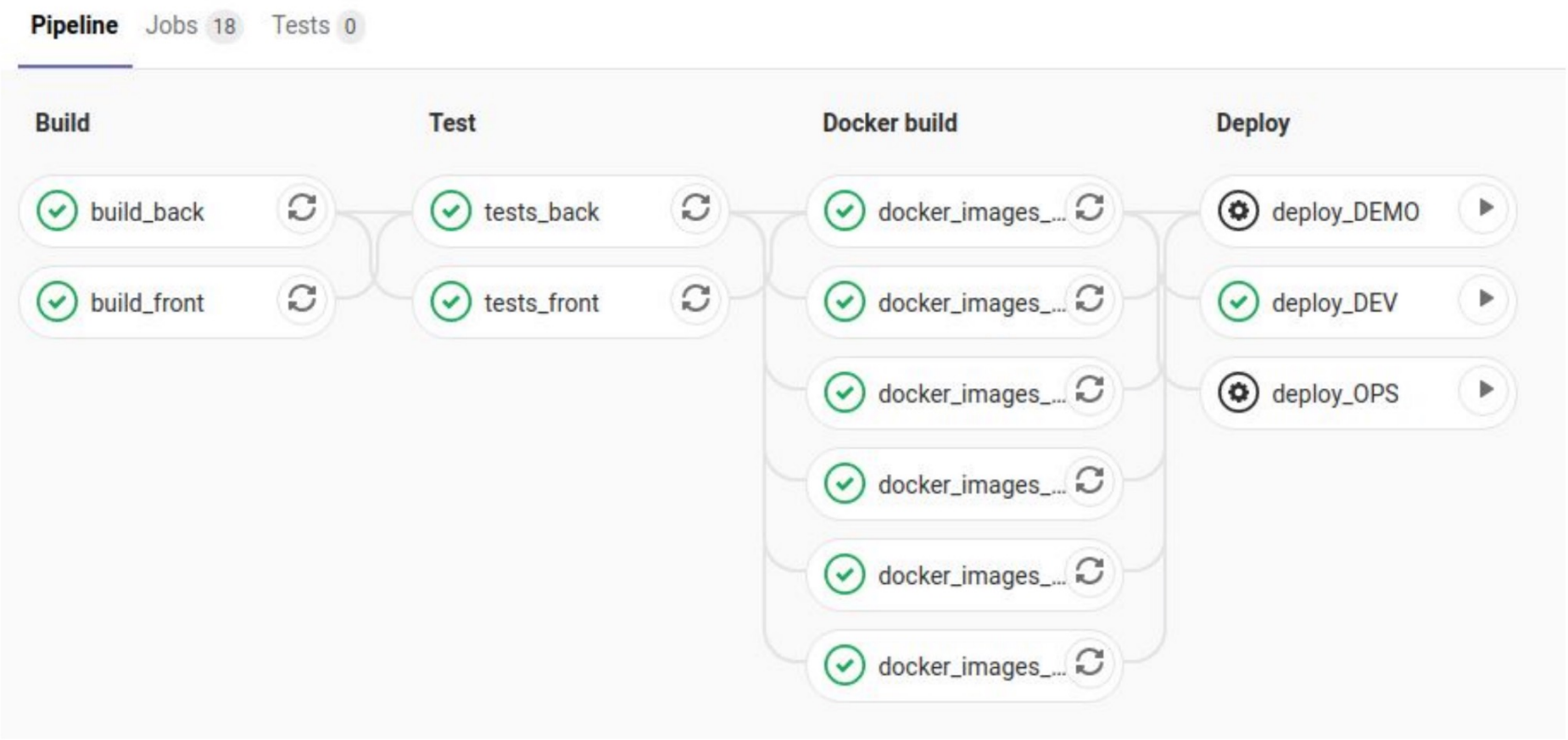
Au vue de ce grand nombre d’étape, on va donc forcément chercher à automatiser certaines choses.

Pour ce faire, on va mettre en place un pipeline afin d’éviter au maximum les actions manuelles

Exemple de Pipeline réel



Exemple de Pipeline réel



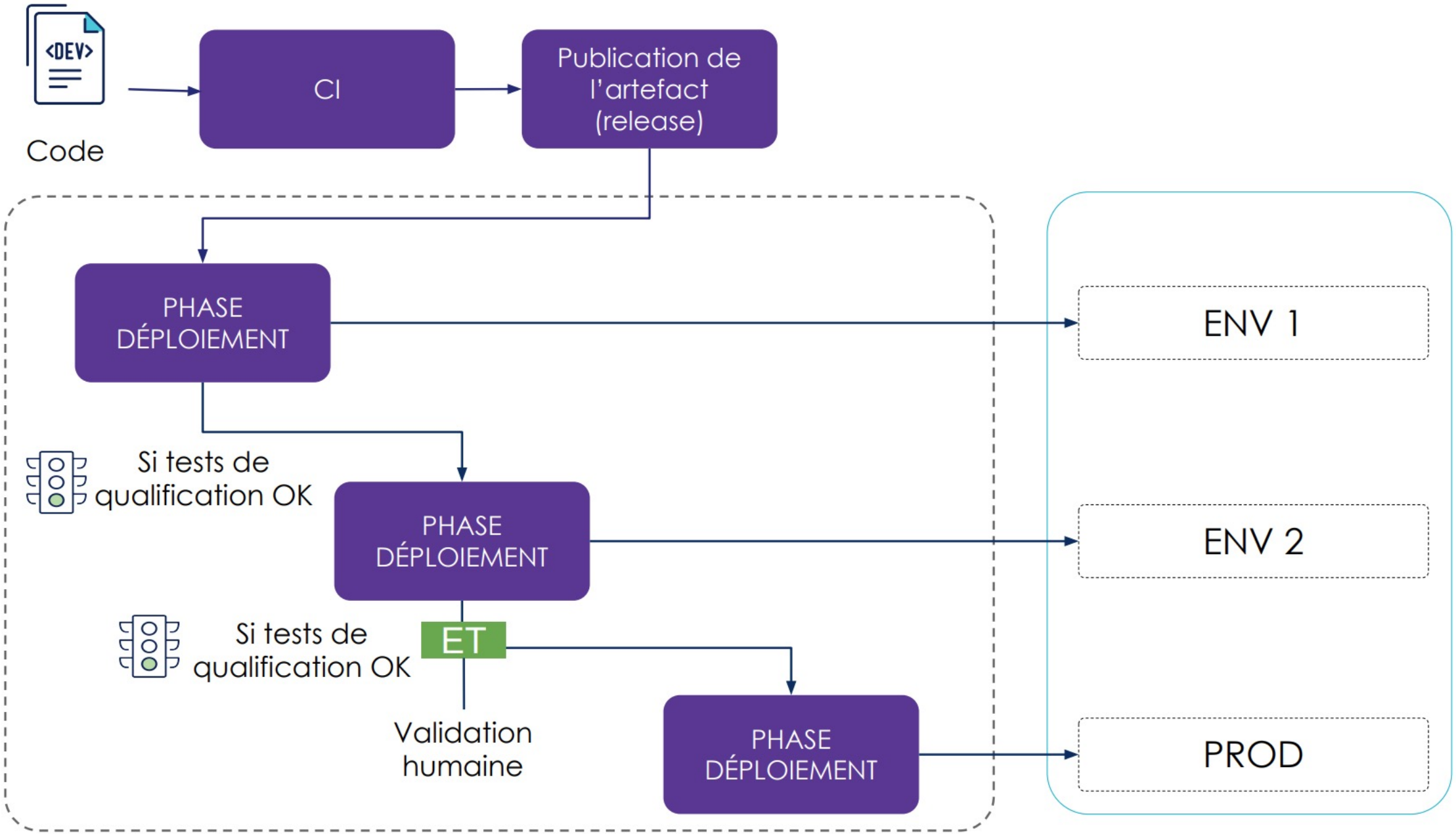
Pipeline – Lié au SCM

Un pipeline se déclenche automatiquement en se basant sur des **événements** :

- Par événement du **gestionnaire de source** (push, tag, merge, création de branche …)
- Déclenchement manuel (clic sur le bouton “Start pipeline”)

Un pipeline est donc étroitement lié **à l’utilisation que l’on fait de son gestionnaire de sources :**

- Workflow et stratégie de branching (ex : gitflow)
- Stratégie de tagging


<DEV>
Code
CI
Publication de l'artefact (release)
PHASE DÉPLOIEMENT
Si tests de qualification OK
PHASE DÉPLOIEMENT
Si tests de qualification OK
ET
Validation humaine
PHASE DÉPLOIEMENT
ENV 1
ENV 2
PROD

Les trois règles de la construction de pipeline

Optimiser la boucle de feedback en exécutant les tests les plus rapides en premier

Ne pas voir le pipeline comme un objet inaltérable

Adapter le pipeline au projet concerné

Un **pipeline** qui n'exécute **aucun tests perd 99 % de son intérêt**.
Il permet juste d'**envoyer des bugs plus vite en production**.